

Challenge 5: GitHub Repository

目标：建立公开、可验证的技术能力。你的 GitHub 不是作业本——它是你的能力证明、作品集、和协作入口。

一、挑战目标

创建一个 GitHub repository，包含**实际项目**、**完整文档**、**使用说明**，让任何人都能看懂、复现、使用。

C5 在 Elite20 的成长路径中处于“生产能力”阶段，和 C4（技能）、C6（Web App）构成一条流水线：

C4 技能 → C5 把技能变成可协作的开源项目 → C6 把项目部署成可用的产品

你可以把 C5 理解为：**给你的 C4 技能一个“家”**——一个别人能找到、能理解、能贡献的地方。

二、为什么是 GitHub

不是因为 GitHub 是唯一选择，而是因为它训练三件事：

能力	GitHub 怎么训练
结构化表达	README 强迫你用别人能懂的方式描述你的项目
可复现性	别人 clone 下来必须能跑，否则就是失败
协作习惯	Issues、PR、版本管理——这是真实工程世界的基本功

即使你不是程序员，一个组织良好的 GitHub repo（比如存放 prompt 模板、workflow 文档、数据集）也比任何简历上的一行字更有说服力。

三、必须满足的要求

3.1 Repo 必须包含

内容	说明	必须/可选
README.md	完整项目说明（见下方模板）	必须
实际项目代码或内容	可运行的代码 / 可安装的 skill / 可用的 prompt 集 / 数据集	必须
使用说明	安装步骤、运行命令、输入输出示例	必须
License	开源协议（推荐 MIT 或 Apache 2.0）	必须
.gitignore	排除不必要的文件	必须
AI 生成日志	AI_LOG.md — 见 Elite20 第二原则	必须
拿来说明	ATTRIBUTION.md — 见拿来主义原则	必须
示例 / Demo	截图、GIF、示例输入输出	推荐
CI/CD	GitHub Actions 自动测试	可选加分
CHANGELOG	版本更新记录	可选加分
CONTRIBUTING.md	协作贡献指南	可选加分

3.2 README.md 标准模板

你的 README 必须包含以下结构（可以用 AI 生成初稿，但必须审校确保准确）：

```
# 项目名称

一句话描述：输入____，输出____。

## 解决什么问题

为什么需要这个项目？没有它之前怎么做？有了它之后怎么做？

## 快速开始

### 安装
```bash
git clone https://github.com/你的用户名/项目名.git
cd 项目名
安装依赖的命令
```
```

```
### 使用
```bash
最简单的使用命令
```
```

```
## 示例
```

输入:

```
> (示例输入)
```

输出:

```
> (示例输出)
```

(或截图 / GIF)

```
## 项目结构
```

```
```
```

项目名/

```
|— README.md
|— src/ (源码)
|— docs/ (文档)
|— examples/ (示例)
|— tests/ (测试)
```

```
```
```

```
## 技术栈
```

- 语言: Python 3.11
- 框架: ...
- AI 集成: Claude API / OpenAI / ...

```
## AI 生成说明
```

本项目使用 AI 辅助开发, 详见 [AI_LOG.md] (./AI_LOG.md)。

```
## 借鉴来源
```

本项目参考了以下来源, 详见 [ATtribution.md] (./ATtribution.md)。

```
## License
```

MIT

四、项目类型（任选方向）

不局限于编程项目。以下都是有效的 C5 提交：

| 类型 | 示例 | README 重点 |
|---------------------|-----------------------|------------------------|
| Skill 包仓库 | 把你的 C4 技能打包成可安装的 repo | 安装命令、触发方式、测试方法 |
| Prompt 模板集 | 一组经过验证的 prompt，按场景分类 | 每个 prompt 的使用场景、示例输入输出 |
| 自动化脚本 | 数据处理 / 文档转换 / 批量操作 | 依赖安装、运行命令、输入输出格式 |
| AI Agent / Workflow | 多步骤 AI pipeline | 架构图、每一步的输入输出、配置说明 |
| 数据集 + 分析 | 收集整理的数据 + 分析代码 | 数据来源、字段说明、分析方法 |
| 文档项目 | 教程、翻译、知识库 | 目录结构、贡献方式、内容索引 |
| Web App 源码 | C6 产品的源码（C5 和 C6 联动） | 部署说明、环境变量、API 文档 |

五、两个真实示例

以下是本次 Elite20 对话中实际创建的两个项目，展示了 Skill → GitHub Repo 的转化路径。

示例 1：wechat-doc-mapper → GitHub Repo

如果要把 wechat-doc-mapper 发布为 GitHub repo，它会长这样：

| | |
|-----------------------------|---------------------------|
| wechat-doc-mapper/ | |
| ├─ README.md | ← 项目说明（一句话：扫描文件夹，映射到挑战） |
| ├─ LICENSE | ← MIT |
| ├─ AI_LOG.md | ← AI 生成日志 |
| ├─ CONTRIBUTION.md | ← 拿来说明 |
| ├─ .gitignore | |
| ├─ SKILL.md | ← Claude Skill 主指令（可直接安装） |
| ├─ scripts/ | |
| │ └─ wechat_doc_mapper.py | ← 核心扫描/映射/Excel生成脚本 |
| ├─ references/ | |
| │ └─ challenges.yaml | ← C1-C7 挑战注册表 |
| ├─ examples/ | |

```
|   |— sample_folder/           ← 测试用的模拟文件夹
|   |— sample_output.xlsx      ← 示例输出
|— tests/
|   |— test_mapper.py          ← 自动化测试
```

README 核心段落：

wechat-doc-mapper — 扫描微信同步文件夹，自动识别每个文件的作者、格式、标题和用途，映射到 Elite20 挑战（C1-C7），输出 Markdown 表格 + 4 页 Excel（含缺口分析）。

输入：文件夹路径 输出：按作者×挑战分组的清单 + .xlsx 文件

拿来说明（ATTRIBUTION.md）要点：

- 文件类型分发逻辑参考了 Claude 内置的 `file-reading` skill
- Excel 格式规范参考了 Claude 内置的 `xlsx` skill
- 命名规范解析的正则表达式参考了 Python `pathlib` 文档

示例 2：skill-explainer → GitHub Repo

```
skill-explainer/
|— README.md
|— LICENSE
|— AI_LOG.md
|— ATTRIBUTION.md
|— .gitignore
|— SKILL.md                ← 纯指令型技能（无脚本）
|— references/
|   |— rubric.md           ← D1-D4 评审维度标准
|— examples/
|   |— sample_report.md    ← wechat-doc-mapper 的 X-ray 分析报告
```

README 核心段落：

skill-explainer — 输入任意 Claude Skill（.skill 文件或目录），输出一份 7 节结构化分析报告：身份、用途、架构、参考文件、使用指南、四维评审（D1描述质量/D2结构完整/D3命名组织/D4可教性）、改进建议。

输入：.skill 文件 / 目录路径 / 已安装技能名 输出：Markdown 分析报告

拿来说明要点：

- 评审维度（D1-D4）参考了 Claude `skill-creator` 的质量标准
- 报告模板结构参考了 `code review` 的标准实践

- “Skill 三层加载”的说明直接复用了 Anthropic 的 skill 开发文档

六、与 C4、C6 的联动

C5 不是孤立的——它是 C4 → C6 流水线的中间节点：

| 从哪来 | C5 做什么 | 到哪去 |
|-------------------|--------------------------------|------------------|
| C4 技能 → | 把技能包装成可协作的开源项目 | → C6 Web App |
| 你在 C4 创建了一个 skill | 在 C5 给它加上 README、测试、示例、License | 在 C6 把它部署成可访问的产品 |

一个项目可以同时提交 **C4 + C5 + C6**，只要各自满足各自的要求并分别附 AI 日志。

fork 别人的 C5 也是有效的 C5 提交——fork → 改进 → 提交 PR → 在你自己的 fork 上写明来说明。这完全符合拿来主义原则，而且对被 fork 的人来说也是 C4 加分（被使用次数 +1）。

七、AI-First 在 C5 的应用

GitHub 项目必须使用 AI 辅助开发。AI_LOG.md 需记录：

| 阶段 | AI 怎么用 | 示例 |
|-------|--------------------------------------|-----------------------------|
| 项目初始化 | 用 AI 生成 repo 骨架、README 模板、.gitignore | “用 Claude 生成 Python 项目标准结构” |
| 代码编写 | 用 AI 辅助写代码、写测试、debug | “用 Copilot 生成核心函数，手动修正边界情况” |
| 文档撰写 | 用 AI 生成 README 初稿、API 文档 | “用 Claude 基于代码自动生成使用文档” |
| 代码审查 | 用 AI review 自己的代码 | “让 Claude 审查安全问题和性能瓶颈” |

手动步骤的反向举证同样适用： 如果你手动写了某段代码而不是用 AI 生成，请在 AI_LOG.md 中说明原因。

八、评判标准

| 权重 | 指标 | 怎么衡量 |
|--------|------|-----------------------------------|
| 🏆 1 高 | 实用性 | 这个项目解决了一个真实的问题吗？别人会用吗？ |
| 🏆 1 高 | 可复现性 | 别人 clone 下来，按 README 操作，能跑起来吗？ |
| 🏆 2 中 | 代码质量 | 结构清晰？命名规范？有注释？有错误处理？ |
| 🏆 2 中 | 文档质量 | README 完整？示例充分？AI 日志和拿来说明都有？ |
| 🏆 3 加分 | 社区验证 | 被 star / fork / 使用 / 被别人在 C4 中引用 |
| 🏆 3 加分 | 持续迭代 | 有多次 commit？有 CHANGELOG？有响应 Issue？ |

最关键的一条： 别人 `git clone` 之后，**5 分钟内能跑起来**。如果做不到，说明你的文档还不够好。

九、提交内容

| 文件 | 命名格式 | 说明 |
|-----------|--------------------|---------------------------------------|
| Repo 链接 | 姓名_c5_repo链接.md | GitHub URL + 一句话项目说明 |
| README 截图 | 姓名_c5_readme截图.png | 可选，展示 README 渲染效果 |
| ⚡ AI 日志 | 姓名_c5_AI日志.md | 开发过程中的 AI 使用记录（也放在 repo 内的 AI_LOG.md） |
| 拿来说明 | 姓名_c5_拿来说明.md | 借鉴来源（也放在 repo 内的 CONTRIBUTION.md） |

提交到群里时附一句话：

"这是我的 C5 GitHub repo: <https://github.com/xxx/yyy> — 输入_____就能得到_____, 欢迎 star、fork、提 Issue! "

十、从零开始的最小路径

如果你从来没用过 GitHub，以下是最快的起步路径（全程可用 AI 辅助）：

Step 1: 注册 GitHub 账号 (github.com)

Step 2: 让 AI 帮你生成项目结构
→ "帮我创建一个 Python 项目的标准 GitHub repo 结构"

Step 3: 把你的 C4 技能文件放进去

Step 4: 让 AI 帮你写 README
→ "根据这些文件帮我写一个完整的 README.md"

Step 5: 推送到 GitHub
→ git init → git add → git commit → git push
(不会用 git? 让 AI 一步一步教你)

Step 6: 在群里分享链接

从零到发布，用 AI 辅助，**30 分钟完全可以做到。**

十一、检查清单

提交前自查：

- ☐ Repo 是 public (公开的)
- ☐ README.md 包含：一句话说明、安装步骤、使用示例、项目结构
- ☐ 别人 clone 后 5 分钟内能跑起来
- ☐ 有 LICENSE 文件
- ☐ 有 .gitignore (不包含不必要的文件)
- ☐ 有 AI_LOG.md (AI 使用日志)
- ☐ 有 CONTRIBUTION.md (拿来说明)
- ☐ 至少有一个真实的使用示例
- ☐ 群内已分享链接并附一句话说明

 **C5 的本质：把能力从“我电脑上的文件”变成“全世界都能用的项目”。**

代码放在本地 = 只对你有用。代码放在 GitHub = 对所有人有用。这就是从个人能力到公共价值的跃迁。